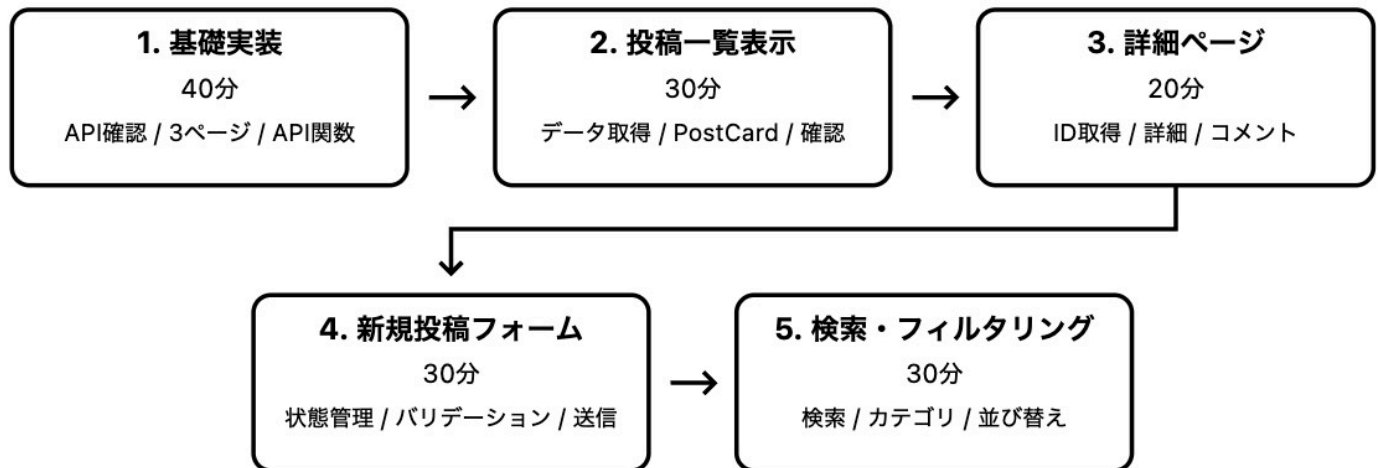


モジュール1 フロントエンド - 実装解説 (React中心)

実装の流れ(推奨)



最初の40分：基礎実装

時間	ステップ	作業内容	使用する技術
5分	APIレスポンス確認	API構造を把握。フィールド名とデータ形式を確認。	Postman
30分	基本ページ作成	3つのページファイル作成。ルーティング設定。基本レイアウト配置。	BrowserRouter Routes Route JSX
5分	API関数の実装	投稿とカテゴリデータ取得関数作成。	fetch async/await

次の30分：投稿一覧表示

時間	ステップ	作業内容	使用する技術
10分	データ取得の実装	ページ読み込み時にデータ取得。	useEffect useState
15分	投稿コンポーネント作成と一覧実装	PostCardコンポーネント作成。投稿一覧表示。詳細ページリンク設定。	コンポーネント作成 map関数 Link
5分	動作確認	表示確認。ページ遷移確認。基本スタイリング。	CSS

次の20分：詳細ページ

時間	ステップ	作業内容	使用する技術
10分	基本データ取得	URLからID取得。投稿データ取得。	useParams useEffect fetch async/await
5分	投稿詳細コンポーネント表示	PostCardコンポーネント再利用。投稿内容追加表示。	PostCard再利用 条件分岐（&&）
5分	コメント表示と戻るボタン	コメント一覧表示。戻るボタン配置。	map関数 useNavigate 条件分岐

次の30分：新規投稿フォーム

時間	ステップ	作業内容	使用する技術
10分	フォーム状態管理	入力フィールドのstate定義。カテゴリデータ取得。	useState onChange
10分	バリデーション実装	必須項目チェック。フォーム有効性判定。送信ボタン制御。	required属性 trim() disabled
10分	送信処理の実装	API送信処理。送信中状態管理。成功時ページ遷移。	fetch JSON.stringify useNavigate async/await

残り30分：検索・フィルタリング機能

時間	ステップ	作業内容	使用する技術
10分	検索機能の実装	検索用state定義。検索入力フィールド作成。リアルタイム検索ロジック。	useState onChange filter includes toLowerCase
10分	カテゴリフィルタの実装	カテゴリ選択用state定義。カテゴリタブUI作成。フィルタリング処理。	useState onClick filter 条件分岐
10分	並び替え機能の実装	並び替え用state定義。切り替えUI作成。ソート処理。	useState sort onClick Date

基礎実装

API関数の実装 (fetchPosts)

JavaScript

```
// API関数：投稿データを取得する関数
const fetchPosts = async () => {
  const response = await fetch('/api/posts') // API呼び出し
  return response.json() // JSONに変換
}
```

投稿一覧表示

データ取得とPostCardコンポーネント (useEffect, PostCard)

JavaScript

```
// 初期データ取得：ページ読み込み時にデータ取得
useEffect(() => {
  const loadData = async () => {
    const posts = await fetchPosts() // 投稿データ取得
    setPosts(posts) // stateに保存
  }
  loadData() // 関数実行
}, []) // 初回のみ
```

JavaScript

```
// PostCardコンポーネント：投稿カード表示コンポーネント
const PostCard = ({ post }) => {
  return (
    <Link to={` /posts/${post.id}`} > /* 詳細ページリンク */
    <div>
      <span>{post.category}</span> /* カテゴリ表示 */
      <h3>{post.title}</h3> /* タイトル表示 */
      <p>投稿者: {post.author}</p> /* 投稿者名 */
      <p>いいね数: {post.likes}</p> /* いいね数 */
      <p>コメント数: {post.commentCount || 0}</p> /* コメント数 */
      {post.imageUrl && <img src={post.imageUrl} alt="" />} /* 画像がある場合のみ表示 */
    </div>
    </Link>
  )
}
```

JavaScript

```
// HomePageでPostCard使用：投稿一覧をPostCardで表示
{posts.map(post => ( // 投稿リストをループ
  <PostCard key={post.id} post={post} /> /* PostCardコンポーネントに投稿データを渡す */
))}
```

新規投稿フォーム

フォーム実装 (useState, handleSubmit, JSX)

JavaScript

```
// フォーム状態管理: 入力フィールドの値管理
const [title, setTitle] = useState('')
const [content, setContent] = useState('')
const [author, setAuthor] = useState('')
const [category, setCategory] = useState('')
const [imageUrl, setImageUrl] = useState('')
const [isSubmitting, setIsSubmitting] = useState(false)

// 必須項目チェック: 全ての必須フィールドが入力されているか確認
const titleOK = title.trim() !== ''
const contentOK = content.trim() !== ''
const authorOK = author.trim() !== ''
const categoryOK = category !== ''
const isValidForm = titleOK && contentOK && authorOK && categoryOK
```

JavaScript

```
// フォーム送信処理: API送信処理関数
const handleSubmit = async (e) => {
  e.preventDefault()
  if (!isValidForm) return

  setIsSubmitting(true)
  const response = await fetch('/api/posts', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title, content, author, category, imageUrl })
  })

  if (response.ok) {
    alert('投稿が作成されました')
    navigate('/')
  }
  setIsSubmitting(false)
}
```

JavaScript

```
// フォームJSX: 入力フィールドと送信ボタン
<form onSubmit={handleSubmit}>
  <input
    value={title}
    onChange={(e) => setTitle(e.target.value)}
    placeholder="タイトル"
    required
  />
  <textarea
```

```

      value={content}
      onChange={(e) => setContent(e.target.value)}
      required
    />
    <input
      value={author}
      onChange={(e) => setAuthor(e.target.value)}
      placeholder="投稿者名"
      required
    />
    <select value={category} onChange={(e) => setCategory(e.target.value)} required>
      <option value="">選択</option>
      {categories.map(cat => (
        <option key={cat.id} value={cat.name}>{cat.name}</option>
      ))}
    </select>
    <input
      value={imageUrl}
      onChange={(e) => setImageUrl(e.target.value)}
      placeholder="画像URL（任意）"
      type="url"
    />
    <button type="submit" disabled={!isFormValid || isSubmitting}>
      {isSubmitting ? '投稿中...' : '投稿する'}
    </button>
  </form>

```

詳細ページ

詳細ページ実装 (useParams, useEffect, JSX)

JavaScript

```

// 詳細ページのデータ取得：URLからID取得して投稿データ取得
const { id } = useParams() // URLからID取得
const navigate = useNavigate() // ナビゲーション用
const [post, setPost] = useState(null) // 投稿データのstate

useEffect(() => {
  const loadPost = async () => {
    const response = await fetch(`/api/posts/${id}`) // 投稿データ取得
    const data = await response.json() // JSON化
    setPost(data) // state保存
  }
  loadPost() // 関数実行
}, [id]) // ID変更時再実行

```

JavaScript

```

// 詳細ページのJSX：PostCardコンポーネントとコメント、ナビゲーションを表示
<div>

```

```

<button onClick={() => navigate('/')}>戻る</button>

{/* PostCardコンポーネントで投稿詳細を表示 */}
{post && <PostCard post={post} />}

<p>{post?.content}</p>

<h3>コメント</h3>
{post?.comments?.length > 0 && (
  post.comments.map(comment => (
    <div key={comment.id}>
      <strong>{comment.author}</strong>: {comment.content}
    </div>
  ))
)}
{post?.comments?.length === 0 && (
  <p>コメントがありません</p>
)}
</div>

```

検索・フィルタリング機能

状態管理 (useState)

```

JavaScript
// 検索・フィルタ・並び替え用状態: 検索ワード、カテゴリ選択、並び順の状態管理
const [searchTerm, setSearchTerm] = useState('') // 検索ワード
const [selectedCategory, setSelectedCategory] = useState('すべて') // カテゴリ
const [sortBy, setSortBy] = useState('new') // 並び順 (new: 新着順, likes: いいね数順)

```

検索機能実装 (filter, input, onChange)

```

JavaScript
// 検索フィルタリング関数: 検索ワードで投稿を絞り込む関数
const searchFilteredPosts = posts.filter(post => {
  if (!searchTerm) return true // 検索ワードがない場合は全部表示

  const titleMatch = post.title.toLowerCase().includes(searchTerm.toLowerCase())
  const contentMatch = post.content.toLowerCase().includes(searchTerm.toLowerCase())

  return titleMatch || contentMatch // タイトルか内容のどちらかにマッチ
})

```

```

JavaScript
// 検索入力フィールド: リアルタイムで投稿を検索する入力フィールド
<input
  value={searchTerm}

```

```
onChange={(e) => setSearchTerm(e.target.value)}
placeholder="検索"
/>
```

カテゴリフィルタ実装 (filter, タブ, onClick)

JavaScript

```
// カテゴリフィルタリング関数: カテゴリで投稿を絞り込む関数
const categoryFilteredPosts = searchFilteredPosts.filter(post => {
  if (selectedCategory === 'すべて') return true // 「すべて」の場合は全部表示
  return post.category === selectedCategory // 選択したカテゴリと一致するか
})
```

JavaScript

```
// カテゴリタブ: カテゴリで投稿をフィルタリングするタブUI
{categories.map(category => (
  <button
    key={category}
    onClick={() => setSelectedCategory(category)}
  >
    {category}
  </button>
))}
```

並び替え機能実装 (sort, Date)

JavaScript

```
// 並び替え関数: フィルタされた投稿をソート
const sortedPosts = [...categoryFilteredPosts].sort((a, b) => {
  if (sortBy === 'likes') {
    return b.likes - a.likes // いいね数の多い順
  }
  return new Date(b.createdAt) - new Date(a.createdAt) // 新しい順
})
```

JavaScript

```
// 並び替えタブ: 新着順・いいね数順の切り替えUI
<button onClick={() => setSortBy('new')}>新着順</button>
<button onClick={() => setSortBy('likes')}>いいね数順</button>
```

JavaScript

```
// 結果表示: フィルタ・ソートされた投稿をPostCardで表示
{sortedPosts.length === 0 && (
  <div>検索結果が見つかりませんでした</div>
```

```
  )}  
  
  {sortedPosts.map(post => (  
    <PostCard key={post.id} post={post} />  
  ))}
```


覚えておくべき知識

React Hooks（重要度順）

- `useState`: 状態管理（入力値、フラグ、配列）
- `useEffect`: 初期データ取得、API呼び出し
- `useParams`: URLパラメータ取得（詳細ページID）
- `useNavigate`: ページ遷移（送信後、戻るボタン）

React Router（基本）

- `<Link to="/path">`: クリック可能なページリンク
- `<Routes>`, `<Route>`: ルーティング設定

JSX必須パターン

- `{array.map()}`: 配列のループ表示
- `{condition && <Component />}`: 条件付き表示
- `{data?.property}`: null安全アクセス
- `onChange={(e) => setState(e.target.value)}`: 入力更新
- `onClick={() => handler()}`: クリック処理
- `onSubmit={handleSubmit}`: フォーム送信

JavaScript頻出パターン

- `async/await`: 非同期API処理
- `fetch('/api/endpoint')`: HTTP通信
- `response.json()`: JSONパース
- `array.filter()`: 検索・絞り込み
- `array.sort()`: 並び替え
- `string.includes()`: 部分一致検索
- `string.trim()`: 前後空白除去
- `[...array]`: 配列コピー（sortで必須）

フォーム・バリデーション

- `required`: HTML5必須項目
- `disabled={!isValid}`: ボタン制御
- `{loading ? '送信中...' : 'ボタン'}`: 状態表示